

Laboratory 2

Ballotta Sofia, Bollengier Alexis, Di Ciano Paolo, Greco Giorgio

EE-512 Applied Biomedical Signal Processing

EPFL, Lausanne Switzerland 23/09/2025

EX01: ECG Signal Enhancement

The objective of this exercise is that you analyse the code provided and make the link with the curse. You have to provide a short report that comments and analyse the results. You can use directly the results or adapt them to you needs.

import the numerical library

```
In [1]: import numpy as np
# import signal processing library
import scipy.signal as sp
# import plotting library
import pylab as py
py.ion()
py.close('all')
```

load the ecg signal

```
In [2]: x = np.genfromtxt('ecg.dat')
# sampling frequency of the signal is 500 Hz
fs = 500
# generate corresponding time vector
t = np.arange(len(x))/fs
```

The signal is an ECG signal with visible PQRST complex. If you zoom on the signal plot you can see that there is a 50Hz perturbation due to the power network. The objective is to remove this component without altering the PQRST complex. Several filtering techniques are used. Comment the advantages and disadvantages.

Plot time signal and FFT.

Q: Comment the figures.

By observing the plots we can see the power line interference at 50 Hz. From the FFT a peak at 50 Hz is clearly visible. Different filtering techniques can be used to remove this type of interference. For example an IIR notch filter would be good option.

Compute the FFT of the signal

```
In [3]: x_fft = np.fft.fft(x)
# Determine the frequency scale
f_fft = np.arange(len(x_fft))/len(x_fft)*fs
```

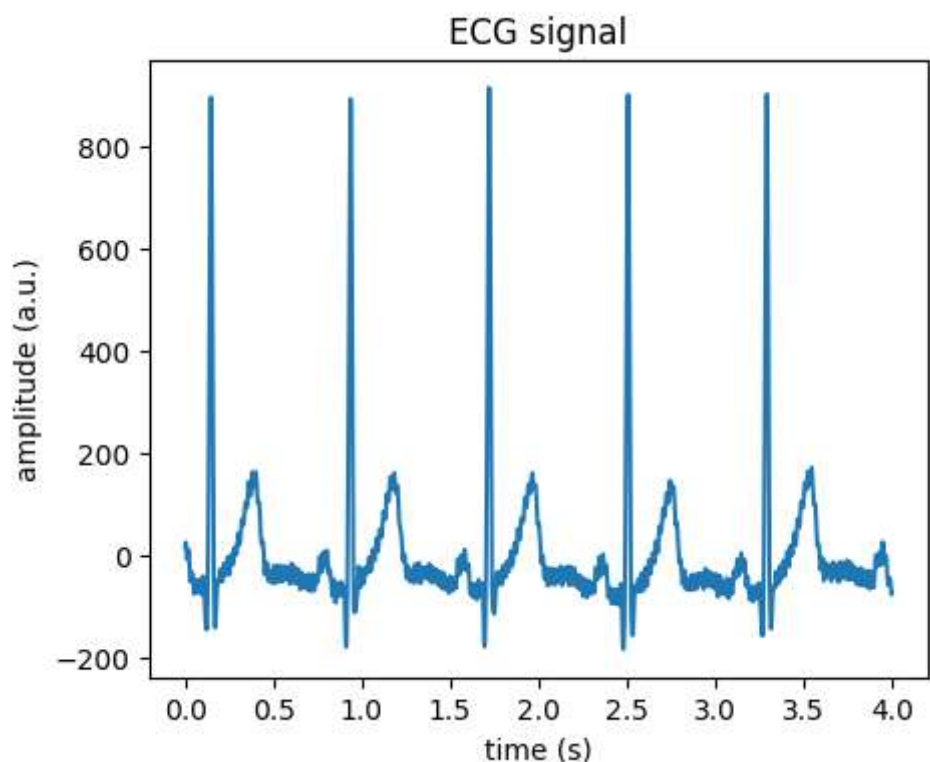
plot the signal

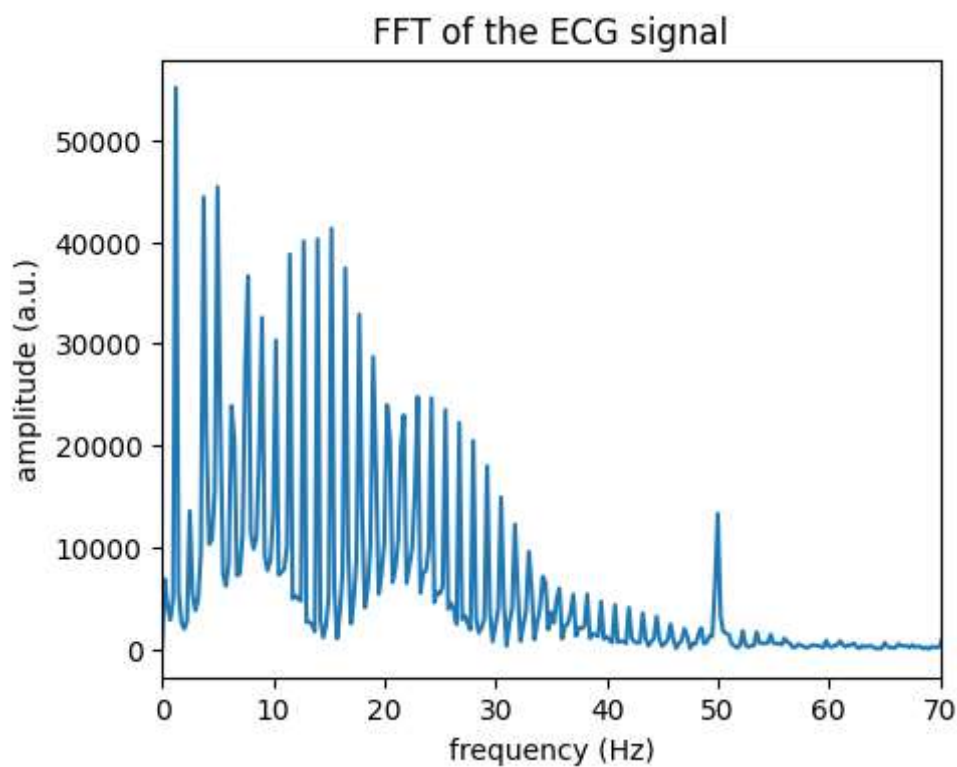
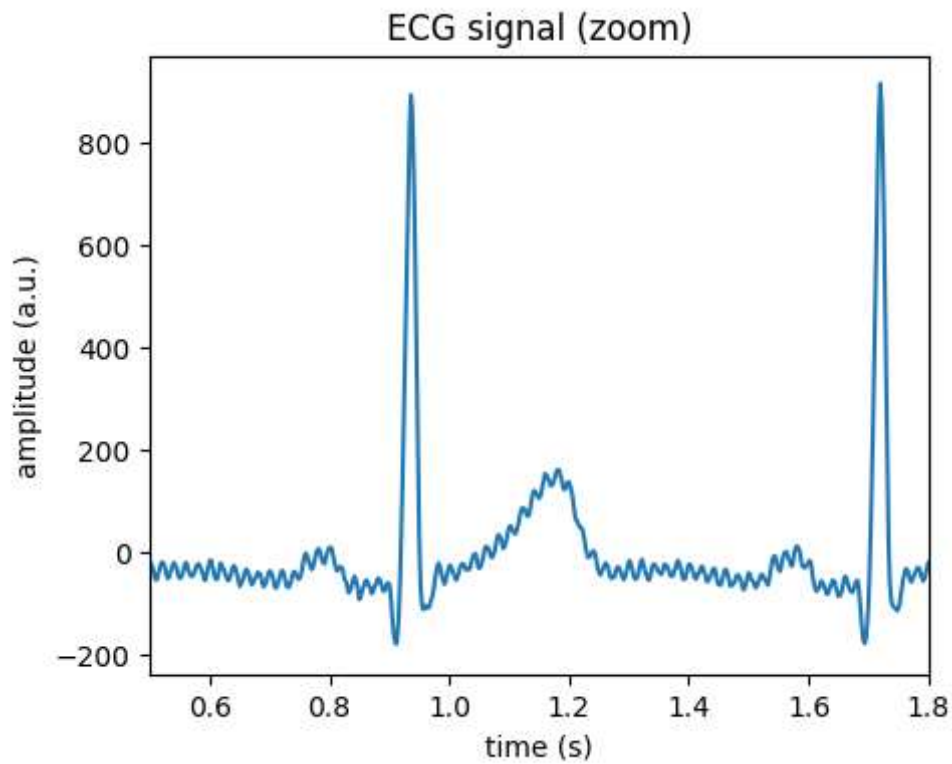
```
In [4]: # Figure 1: ECG signal (full)
py.figure(1, figsize=[5,4])
py.clf()
py.plot(t, x)
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('ECG signal')

# Figure 2: ECG signal (zoom)
py.figure(2, figsize=[5,4])
py.clf()
py.plot(t, x)
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('ECG signal (zoom)')
py.xlim(0.5, 1.8)

# Figure 3: FFT of the ECG signal
py.figure(3, figsize=[5,4])
py.clf()
py.plot(f_fft, abs(x_fft))
py.xlabel('frequency (Hz)')
py.ylabel('amplitude (a.u.)')
py.title('FFT of the ECG signal')
py.xlim(0, 70)
```

Out[4]: (0.0, 70.0)





IIR filter:

Define a filter with a pass-band up to 35 Hz and a stop band from 50Hz.

Maximum attenuation in passband 3 dB

Minimum attenuation in stopband 40 dB

Q: Comment the results (distorsion of the PQRST, delay, ...)

We observe the PQRST complex shifted and slightly distorted. The time-shift of the signal is due to the delay introduced by the causal filter. The distortion is mainly due to 2 effects: 1. While Butterworth filters guarantee maximally flat passband, the transition

band is gradual, hence frequencies near the cutoff are attenuated. 2. Because the filter has a nonlinear phase, different frequency components are shifted in time differently. This means the sharp morphology of the ECG does not reconstruct perfectly after filtering.

Q: Based on the FFT spectrum comment the selection of the pass and stop band frequencies.

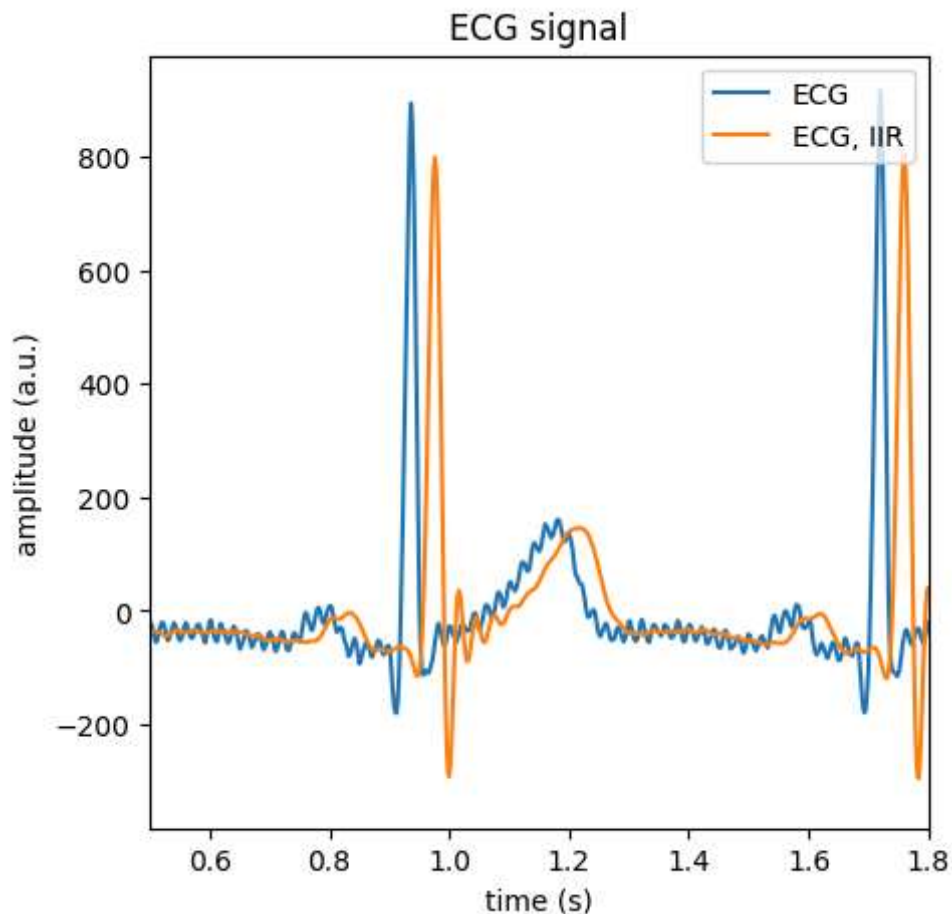
From the FFT of the ECG signal, it can be seen that most of the useful energy is concentrated below 35–40 Hz. The P and T waves occupy the lower frequencies (≈ 10 Hz), while the QRS complex, which represents the fastest part of the signal, contains energy mainly between 10 and 40 Hz, with significant contributions around 15–30 Hz and residual components up to about 40 Hz. The distinct peak at 50 Hz does not belong to the cardiac signal: it is clearly power line interference. The choice of a pass-band up to 35 Hz is therefore consistent: it preserves most of the physiological information (P, QRS, T) while reducing high-frequency noise. Defining the stop-band at 50 Hz with at least 40 dB attenuation is aimed at suppressing the power line disturbance visible in the spectrum. The transition between 35 and 50 Hz is fairly wide (15 Hz), so it can be implemented without excessively high filter orders, even with an IIR. However, in this region there are still components of the QRS complex: a cutoff at 35 Hz may slightly attenuate its steepest slopes.

Analogic limit of the passband frequency

```
In [5]: f_pass = 35
# Analogic limit of the stopband frequency
f_stop = 50
# Conversion into Nyquist frequency
f_pass_N = f_pass/fs*2
f_stop_N = f_stop/fs*2
# Max attenuation in passband (dB)
g_pass = 3
# Min attenuation in stopband (dB)
g_stop = 40
# Determine the order and the cutoff frequency of a butterworth filter
ord, wn = sp.buttord(f_pass_N, f_stop_N, g_pass, g_stop)
# Compute the coefficients of the filter
b, a = sp.butter(ord, wn)
# Filter the signal
x_f = sp.lfilter(b, a, x)
```

```
In [6]: py.figure(2, figsize=[5,5])
py.clf()
py.plot(t, x, label='ECG')
py.plot(t, x_f, label='ECG, IIR')
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('ECG signal')
py.legend(loc='upper right')
py.xlim(0.5, 1.8)
```

```
Out[6]: (0.5, 1.8)
```



IIR filter (zero phase):

Use the same filter but apply a zero phase approach.

Q: Comment the results (distorsion of the PQRST, delay, ...)

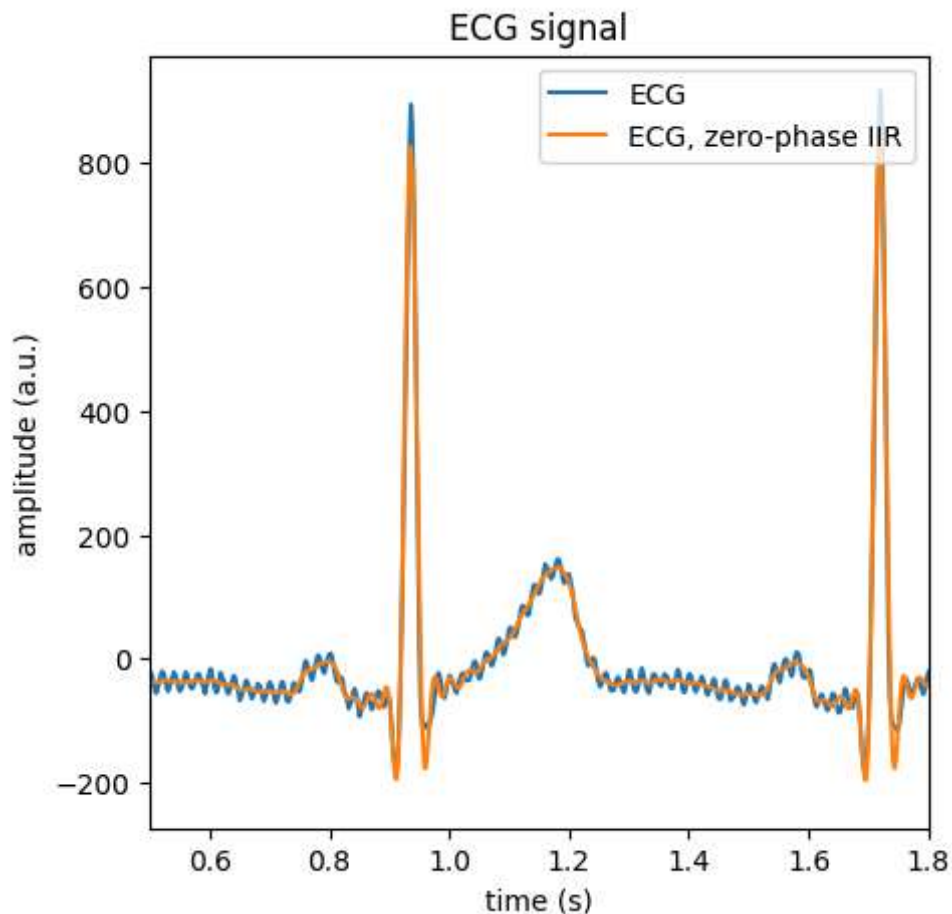
By using a zero-phase filtering approach the group delay is removed, hence obtaining a signal that is no more shifted This approach is only feasible for asynchronous filtering

Filter the signal

```
In [7]: x_f = sp.filtfilt(b ,a, x)
```

```
In [8]: py.figure(3, figsize=[5,5])
py.clf()
py.plot(t ,x, label='ECG')
py.plot(t, x_f, label='ECG, zero-phase IIR')
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('ECG signal')
py.legend(loc='upper right')
py.xlim(0.5, 1.8)
```

```
Out[8]: (0.5, 1.8)
```



Linear phase FIR filter.

Define a FIR filter with the same properties.

Q: Comment the results (distorsion of the PQRST, delay, ...).

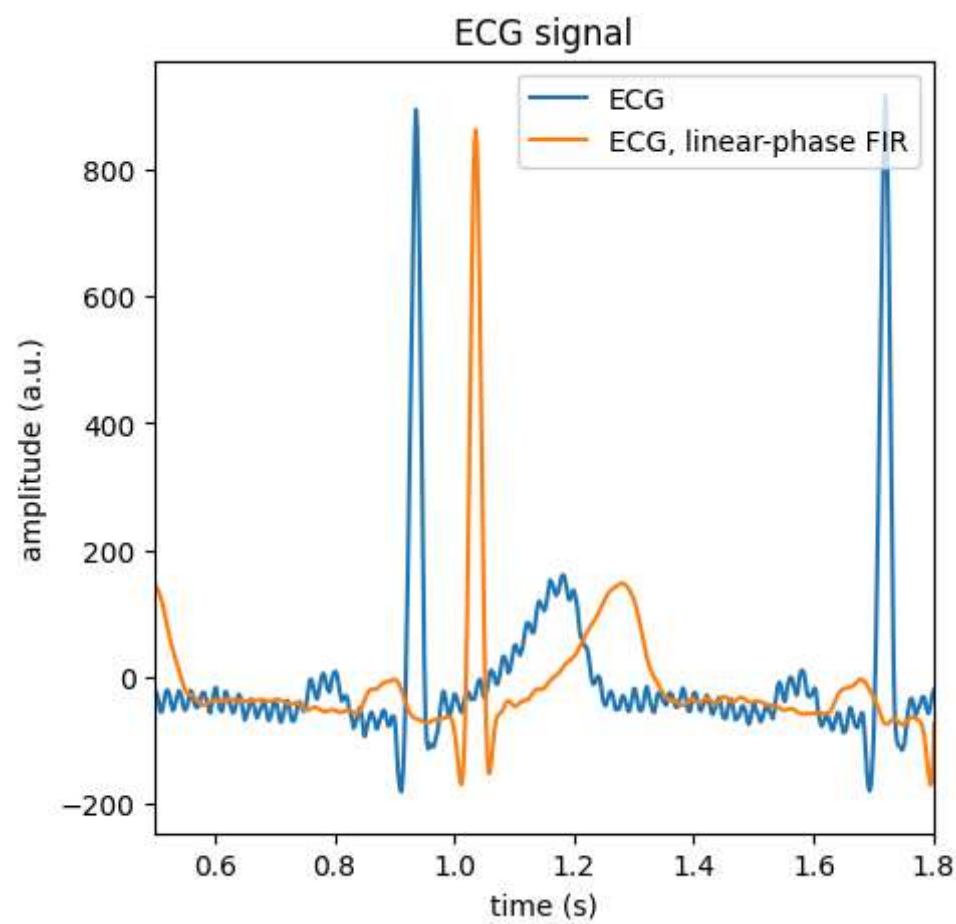
Linear phase filters maintain the true morphology of the ECG by introducing only a constant time delay, without distorting the relative timing of different frequency components. This is why the PQRST complex looks "cleaner" and less distorted, even though the entire waveform is shifted in time.

length of the filter

```
In [9]: l_fir = 101
# compute the filter coefficients using least square approach
b = sp.firls(l_fir, [0, f_pass_N, f_stop_N, 1], [1, 1, 1/100, 1/100])
a = [1]
# filter the signal
x_f = sp.lfilter(b, a, x)
```

```
In [10]: py.figure(4, figsize=[5,5])
py.clf()
py.plot(t, x, label='ECG')
py.plot(t, x_f, label='ECG, linear-phase FIR')
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('ECG signal')
py.legend(loc='upper right')
py.xlim(0.5, 1.8)
```

Out[10]: (0.5, 1.8)



In []:

In []:

In []:

In []:

EX02: Breathing Signal estimation

The objective of this exercise is that you analyse the code provided and make the link with the curse. You have to provide a short report that comments and analyse the results. You can use directly the results or adapt them to you needs.

import the numerical library

```
In [1]: import numpy as np
# import signal processing library
import scipy.signal as sp
# import plotting library
import pylab as py
py.ion()
py.close('all')
```

load the ecg signal

```
In [2]: x = np.genfromtxt('respiration.dat')
# sampling frequency of the signal is 500 Hz
fs = 2
# generate correponding time vector
t = np.arange(len(x))/fs
```

The signal is a measurement of the breathing obtained by inductance plethysmography.

The objective is to estimate the breathing frequency.

The Hilbert transforms permits to estimate the instaneous amplitude and phase of a narrow band signal.

Q: Comment the figures.

We observe the breathing signal and its envelop given by the Hilbert trasform. We observe that the analytic signal does not concide with the envelope of the signal

Q: Why the envelope does no follow the maxima of the signal

The raw breathing signal does not fullfil the requirement of being narrow band. There is no clearly defined carrier frequency, instead the spectrum of the signal is spread.

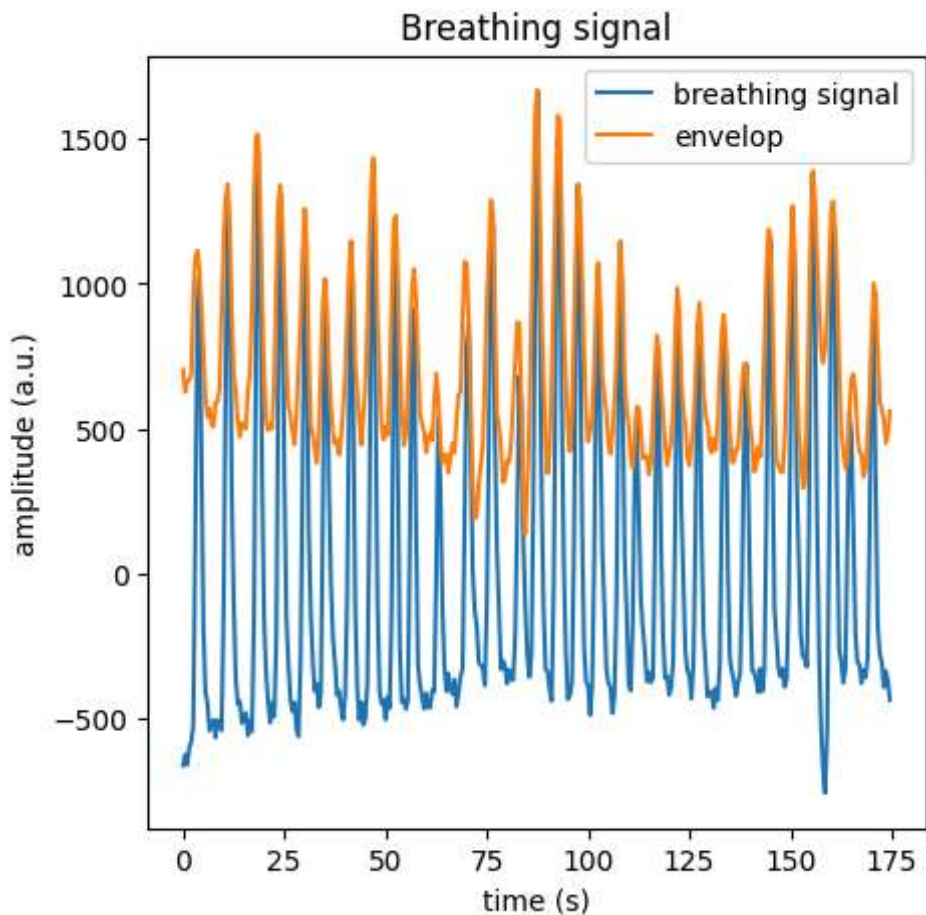
compute the analytical signal of x (Hilbert transform)

```
In [3]: xa = sp.hilbert(x)
```


plot the signal

```
In [4]: py.figure(1, figsize=[5,5])
py.clf()
py.plot(t, x, label='breathing signal')
py.plot(t, np.abs(xa), label='envelop')
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.legend(loc='upper right')
py.title('Breathing signal')
```

Out[4]: Text(0.5, 1.0, 'Breathing signal')



The raw breathing signal does not fulfill the requirement of narrow band.

The normal range of frequency for the breathing is within 0.1 to 0.25 Hz.

The signal is first filtered for this interval.

Q: Comment the figures

In the figures, we see the breathing signal after filtering, with the corresponding analytical signal (constructed via the Hilbert transform) superimposed. After filtering, the signal has become approximately narrowband, meaning its energy is concentrated around a central frequency with only slow variations in amplitude and phase. Because of this narrowband property, the envelope of the analytic signal accurately tracks the true modulation of the breathing cycle. In other words, the smooth curve defined by the analytic signal now follows the oscillatory peaks of the real breathing waveform

Q: How is the estimation of the amplitude envelope.

The filtering removes higher-frequency components that would otherwise interfere and create spurious local maxima, making the envelope visually inconsistent. Once the breathing signal is constrained to a narrow frequency band, the analytic signal captures its amplitude modulation reliably, and the instantaneous amplitude (envelope) appears to follow the maxima of the waveform as expected.

Analogic limit of the passband frequency

```
In [5]: f_pass = np.array([0.1, 0.25])
# Analogic limit of the stopband frequency
f_stop = np.array([0, 0.6])
# Conversion into Nyquist frequency
f_pass_N = f_pass/fs*2
f_stop_N = f_stop/fs*2
# Max attenuation in passband (dB)
g_pass = 3
# Min attenuation in stopband (dB)
g_stop = 40
# Determine the order and the cutoff frequency of a butterworth filter
ord, wn = sp.buttord(f_pass_N, f_stop_N, g_pass, g_stop)
# Compute the coefficients of the filter
b, a = sp.butter(ord, wn, btype='band')
# Filter the signal
x_bp = sp.filtfilt(b, a, x)
```

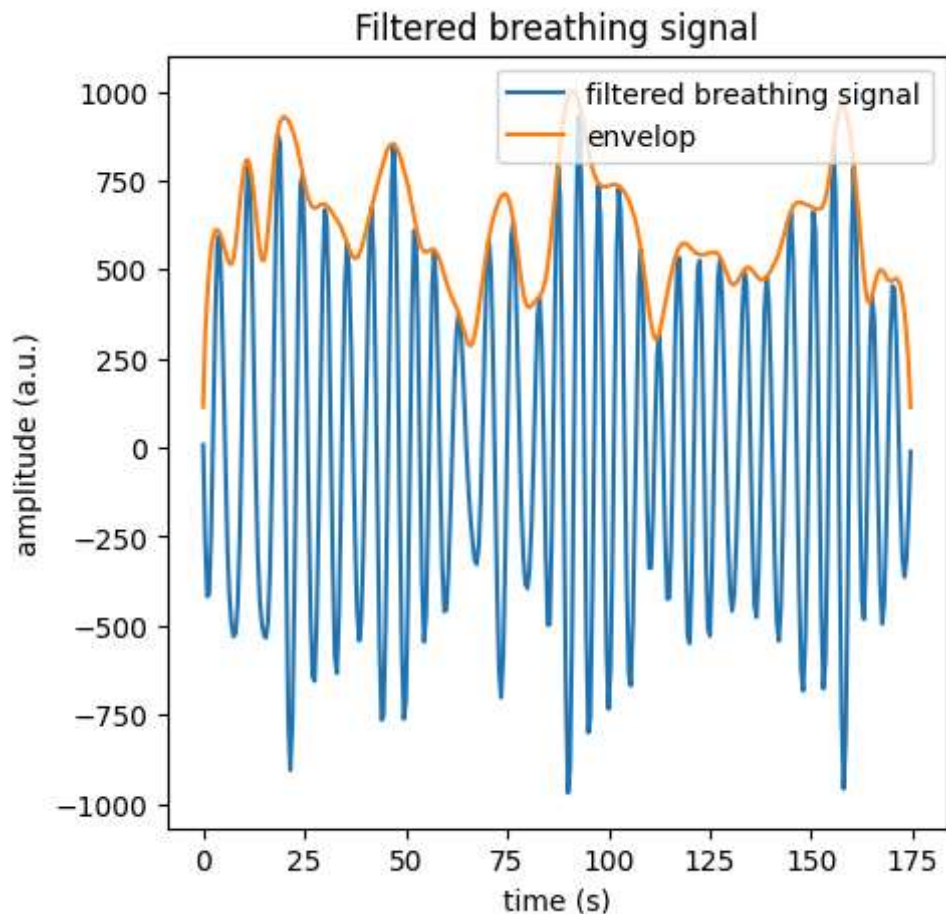
```
c:\Users\alexb\AppData\Local\Programs\Python\Python313\Lib\site-packages\scipy\signal\_filter_design.py:4137: RuntimeWarning: divide by zero encountered in divide
    nat = ((stopb ** 2 - passb[0] * passb[1]) /
```

Compute the Hilbert transform.

```
In [6]: xa = sp.hilbert(x_bp)
```

```
In [7]: py.figure(2, figsize=[5,5])
py.clf()
py.plot(t, x_bp, label='filtered breathing signal')
py.plot(t, np.abs(xa), label='envelop')
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('Filtered breathing signal')
py.legend(loc='upper right')
```

```
Out[7]: <matplotlib.legend.Legend at 0x19e84c102d0>
```



The angle of the Hilbert transform gives the instantaneous phase of the signal.

Q: Comment the figure.

In the first plot we see the instantaneous phase of the analytic signal obtained through the Hilbert transform, computed with `np.angle`. By definition, this function restricts the phase values to the interval between $-\pi$ and π . As the signal evolves over time, whenever the phase increases beyond π or decreases below $-\pi$, it is wrapped back into this range. This produces artificial discontinuities of 2π , so the phase appears as a sawtooth-like curve with sudden jumps rather than a continuous trajectory

Q: What is the role of the `unwrap` function

In the second plot, the function `np.unwrap` has been applied to the signal. This approach detects the discontinuities of 2π and compensates them by adding or subtracting the appropriate multiples of 2π . As a result, the phase no longer resets but evolves smoothly and continuously over time. The unwrapped phase therefore provides a representation of the real progression of the signal's phase.

estimate the instantaneous phase from the Hilbert transform

```
In [8]: phi_xa = np.angle(xa)
# phase is bounded between -pi and pi -> reconstruct continuous signal
phi_xa_unw = np.unwrap(phi_xa)
```

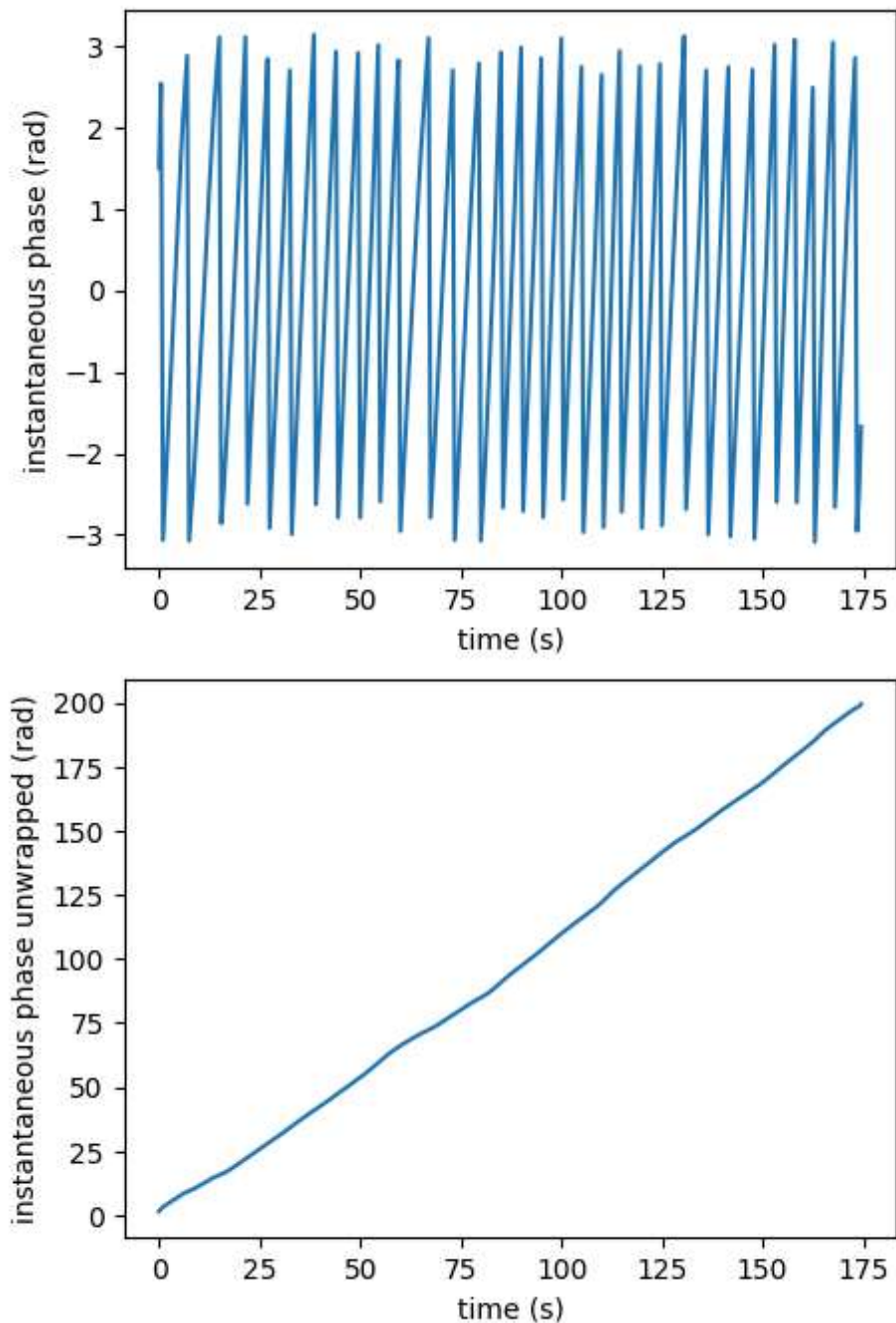
```
In [9]: py.figure(3, figsize=[5, 8])
py.clf()
py.subplot(2,1,1)
```

```

py.plot(t, phi_xa)
py.xlabel('time (s)')
py.ylabel('instantaneous phase (rad)')
py.subplot(2,1,2)
py.plot(t, phi_xa_unw)
py.xlabel('time (s)')
py.ylabel('instantaneous phase unwrapped (rad)')

```

Out[9]: Text(0, 0.5, 'instantaneous phase unwrapped (rad)')



The time derivate of the instaneous phase is the instaneous frequency of the signal.

Q: Comment the figure.

The first figure is the raw breathing signal in the time domain, showing the oscillatory waveform of inhalation and exhalation. Its amplitude varies, and the breathing cycles are clearly visible but irregular in depth and duration. The second figure shows the

instantaneous breathing rate (in breaths per minute, bpm) obtained from the derivative of the unwrapped Hilbert phase. The breathing rate fluctuates over time, mostly between 8 and 14 bpm, which matches the physiological range for resting respiration. The curve captures variability in breathing frequency that may not be obvious from simply looking at the raw signal.

Q: Compare the original waveform with the estimation of the breathing frequency

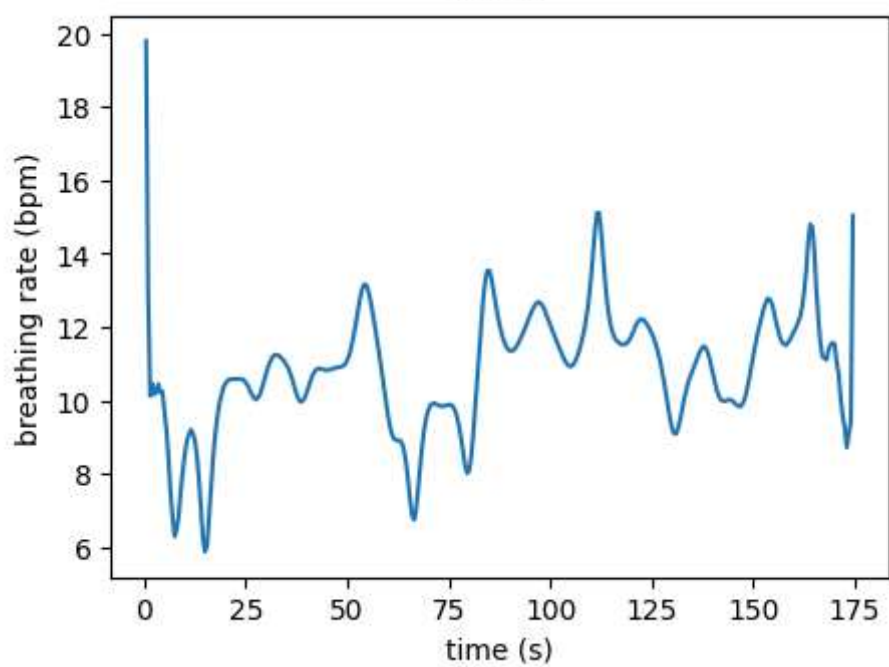
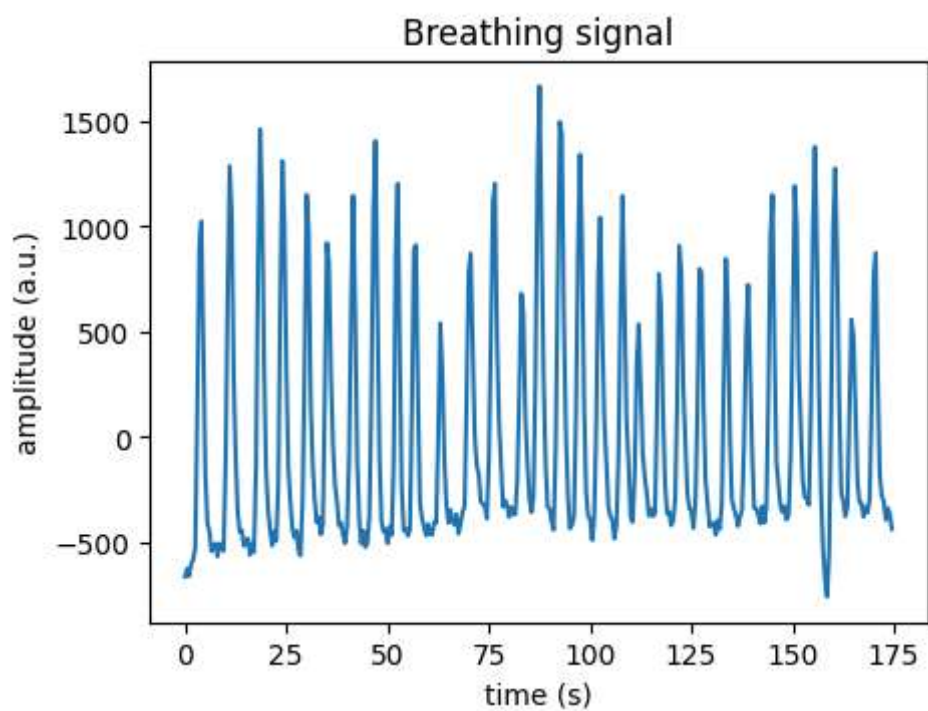
The raw signal provides the shape and amplitude of breathing cycles, but extracting frequency directly from it is not straightforward, especially when the waveform is irregular or noisy. The Hilbert phase method transforms the oscillatory pattern into a smooth, continuous estimate of breathing rate, which makes it much easier to quantify frequency changes. While the waveform shows cycle-to-cycle variations, the frequency estimate condenses this into a time-varying rate curve. This highlights trends (slower/faster breathing) without being affected by amplitude changes.

compute the derivative of the phase (angular frequency).

```
In [10]: d_phi = np.diff(phi_xa_unw)
# convert angular frequency to frequency.
d_phi /= 2*np.pi
# convert digital frequency to analog frequency and in breathing per minute
# (bpm)
d_phi *= fs*60
```

```
In [11]: py.figure(4, figsize=[5,8])
py.clf()
py.subplot(2,1,1)
py.plot(t, x, label='breathing signal')
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('Breathing signal')
py.subplot(2,1,2)
py.plot(t[1:], d_phi)
py.xlabel('time (s)')
py.ylabel('breathing rate (bpm)')
```

```
Out[11]: Text(0, 0.5, 'breathing rate (bpm)')
```



EX03:Hand Washing Detection using Accelerometer Signals

The objective of this exercise is that you analyse the code provided and make the link with the curse. You have to provide a short report that comments and analyse the results. You can use directly the results or adapt them to you needs.

import the numerical library

```
In [1]: import numpy as np
# import signal processing library
import scipy.signal as sp
# import plotting library
import pylab as py
py.ion()
py.close('all')
```

load the ecg signal

```
In [2]: x = np.genfromtxt('accel.dat')
# sampling frequency of the signal is 500 Hz
fs = 40
# generate correponding time vector
t = np.arange(len(x))/fs
```

The signal is an acceleration signal measured at the wrist.

The signal records the acceleration during the hand washing protocol in an hospital. The goal is to detect the hand washing sequence.

The hand washing protocol produced rhythmical movements.

The protocol is known to take place between 20 and 30 seconds.

Plot time signal.

Q: Comment the figure.

The acceleration signal remains relatively steady during the first ~20 seconds, showing only small fluctuations around a constant baseline. At around 20 seconds a marked transition occurs: the amplitude increases sharply and the waveform becomes clearly rhythmic, indicating the start of the hand-washing sequence. Between ~22 and ~30 seconds the activity is sustained, with frequent bursts and pronounced excursions, such as the large drop near 30 seconds, that reflect the vigorous and repetitive strokes typical of the washing protocol. After 30 seconds the signal amplitude decreases and its variability reduces, marking the conclusion of the hand-washing protocol. This temporal

pattern clearly isolates the washing sequence as a distinct, high-energy episode embedded within two other quieter signal segments.

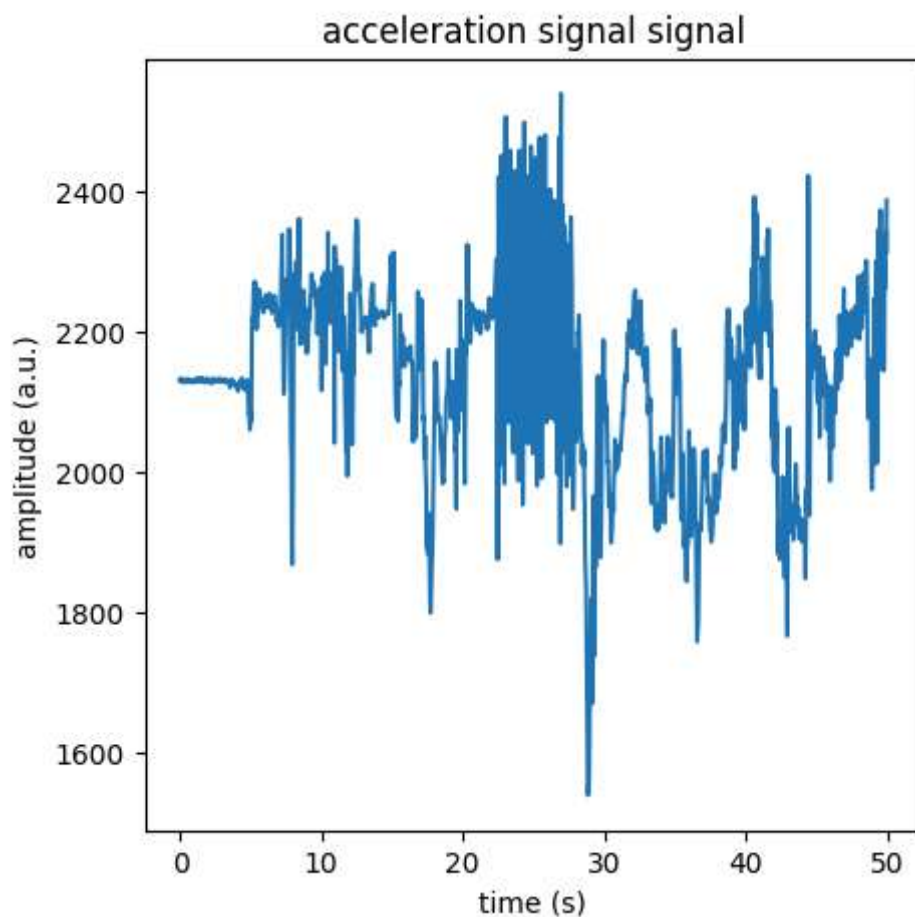
Compute the FFT of the signal

```
In [3]: x_fft = np.fft.fft(x)
# Determine the frequency scale
f_fft = np.arange(len(x_fft))/len(x_fft)*fs
```

plot the signal

```
In [4]: py.figure(1, figsize=[5,5])
py.clf()
py.plot(t, x)
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('acceleration signal signal')
```

```
Out[4]: Text(0.5, 1.0, 'acceleration signal signal')
```



High pass the signal.

Q: Comment the figure.

After applying a high-pass filter at 0.5 Hz, the wrist acceleration signal changes noticeably. The raw signal contained both fast hand movements and slow variations due to posture, sensor noise, or gravity, which made the hand washing sequence harder to see. The filter removes these slow components and highlights the faster, rhythmic ones. In the top plot, between 20 and 30 seconds, the signal shows a clearer oscillatory pattern,

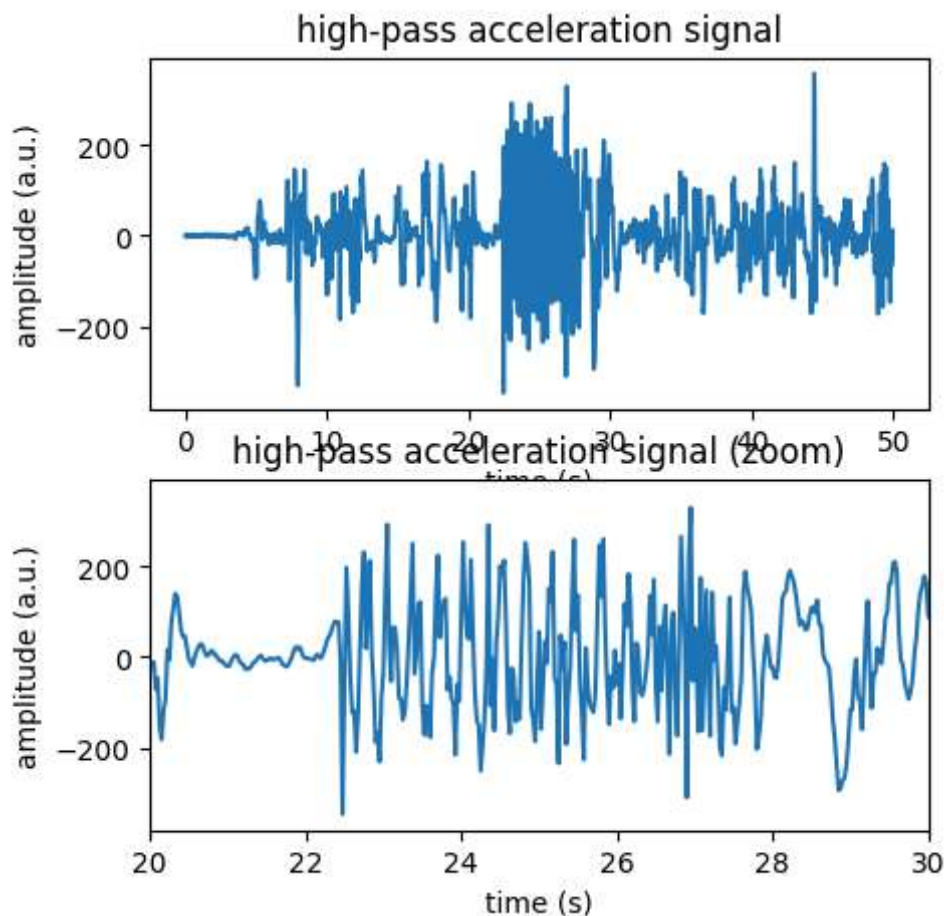
corresponding to hand washing. The zoomed view further emphasizes this periodicity with compact, regular oscillations. In short, the high-pass filter cleans the signal from low-frequency noise and brings out the hand washing rhythm, making it much easier to detect.

high-pass filter with cutoff frequency of 0.5 Hz

```
In [5]: b, a = sp.butter(4, 0.5/fs*2, btype='high')
# zero-phase filtering of the signal
x_hp = sp.filtfilt(b, a, x)
```

```
In [6]: py.figure(2, figsize=[5,5])
py.clf()
py.subplot(2,1,1)
py.plot(t, x_hp)
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('high-pass acceleration signal')
py.subplot(2,1,2)
py.plot(t, x_hp)
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('high-pass acceleration signal (zoom)')
py.xlim(20, 30)
```

Out[6]: (20.0, 30.0)



Band pass the signal between 2.4 and 3.2 Hz.

Q: Based on previous figure, comment the selection of the frequencies.

The band-pass filtered signal between 2.4 and 3.2 Hz represents the dominant oscillatory component of the wrist accelerometer during the handwashing task. This frequency range corresponds to the repetitive wrist and hand movements that occur during scrubbing, rinsing, and rubbing, typically performed at a rate of about 2–3 cycles per second. After filtering, only the components of the signal within this frequency band are preserved, meaning that slow movements (e.g., posture changes, drift, or low-frequency noise) and high-frequency components (e.g., tremors, sensor noise, or sharp micro-motions) are suppressed. What remains is the periodic rhythmic motion of the wrist, which characterizes the core mechanical pattern of handwashing.

Q: Why zero phase filter (filtfilt) is used?

A zero-phase filter such as `filtfilt` is applied to avoid phase distortion. When a causal filter is used in the forward direction only, the output is delayed by the filter's group delay, causing a temporal shift of peaks and relevant features. By filtering the signal forward and backward, this delay is cancelled and the overall phase response becomes zero, ensuring that the morphology and timing of the hand-washing signal are preserved.

Analogic limit of the passband frequency

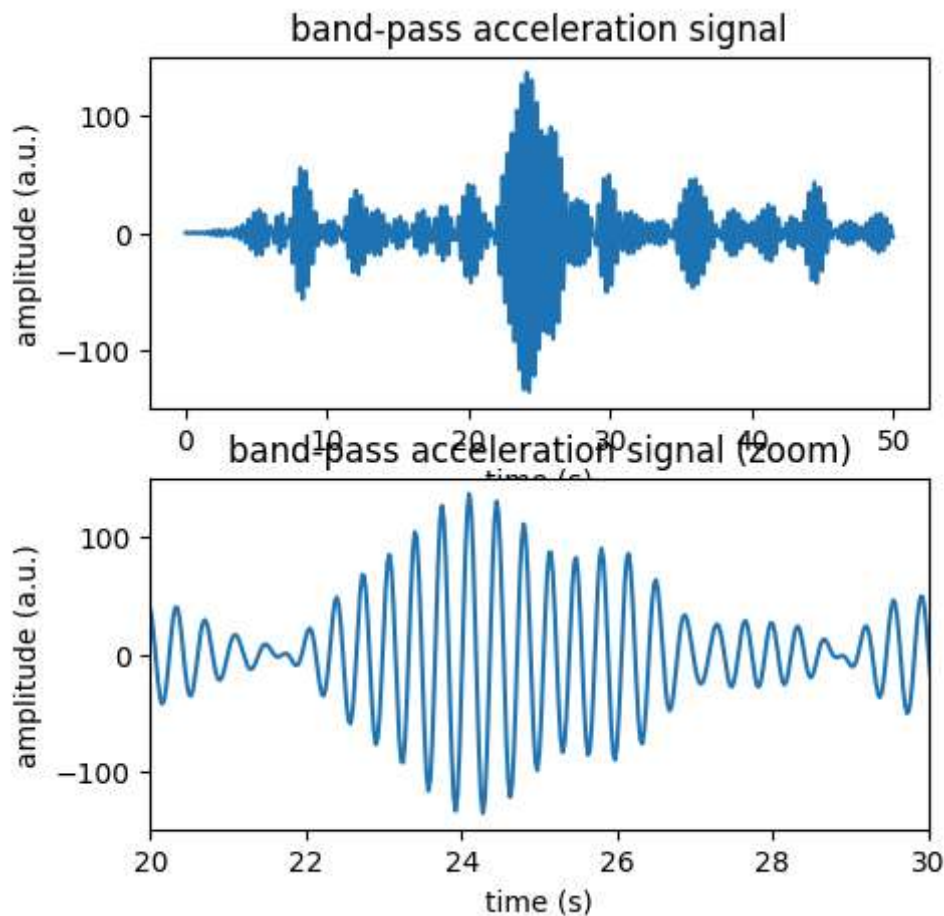
```
In [7]: f_pass = np.array([2.4, 3.2])
# Analogic limit of the stopband frequency
f_stop = np.array([0, 5])
# Conversion into Nyquist frequency
f_pass_N = f_pass/fs*2
f_stop_N = f_stop/fs*2
# Max attenuation in passband (dB)
g_pass = 3
# Min attenuation in stopband (dB)
g_stop = 40
# Determine the order and the cutoff frequency of a butterworth filter
ord, wn = sp.buttord(f_pass_N, f_stop_N, g_pass, g_stop)
# Compute the coefficients of the filter
b, a = sp.butter(ord, wn, btype='band')
# Filter the signal
x_bp = sp.filtfilt(b, a, x_hp)
```

```
c:\Users\alexb\AppData\Local\Programs\Python\Python313\Lib\site-packages\scipy\signal\_filter_design.py:4137: RuntimeWarning: divide by zero encountered in divide
  nat = ((stopb ** 2 - passb[0] * passb[1]) /
```

```
In [8]: py.figure(3, figsize=[5,5])
py.clf()
py.subplot(2,1,1)
py.plot(t, x_bp)
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('band-pass acceleration signal')
py.subplot(2,1,2)
py.plot(t, x_bp)
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
```

```
py.title('band-pass acceleration signal (zoom)')
py.xlim(20, 30)
```

Out[8]: (20.0, 30.0)



Low-pass filter of the power of the band-pass signal.

Q: Why use the power of the acceleration signal?

When the wrist acceleration signal is observed directly, it appears as a waveform oscillating around zero with both positive and negative values. This makes it difficult to assess the intensity of the motor activity, since positive and negative contributions tend to cancel each other out. By computing the power of the signal (i.e., by squaring it), an always positive measure is obtained that directly reflects the energy of the movement. This representation makes it easier to identify periods of greater activity, such as the hand washing sequence, which corresponds to a clear increase in power compared to intervals of rest or minor movements.

Q: How the detection of hand washing is obtained?

The detection process begins with the band-pass filtered acceleration signal, in order to isolate the frequencies typical of the rhythmic hand washing movement. The power of this signal is then computed, providing an instantaneous measure of movement intensity. Since the raw power remains noisy, a low-pass filter is applied to smooth the trend and produce a more regular curve. In this filtered curve, hand washing appears as a clear and localized peak. At this stage, a threshold is defined: when the power exceeds the threshold, the system detects that hand washing is taking place; when the power falls

below the threshold, the activity is considered finished. In the plot, this detection is represented by the orange line, which rises clearly between 20 and 30 seconds, exactly during the interval of hand washing

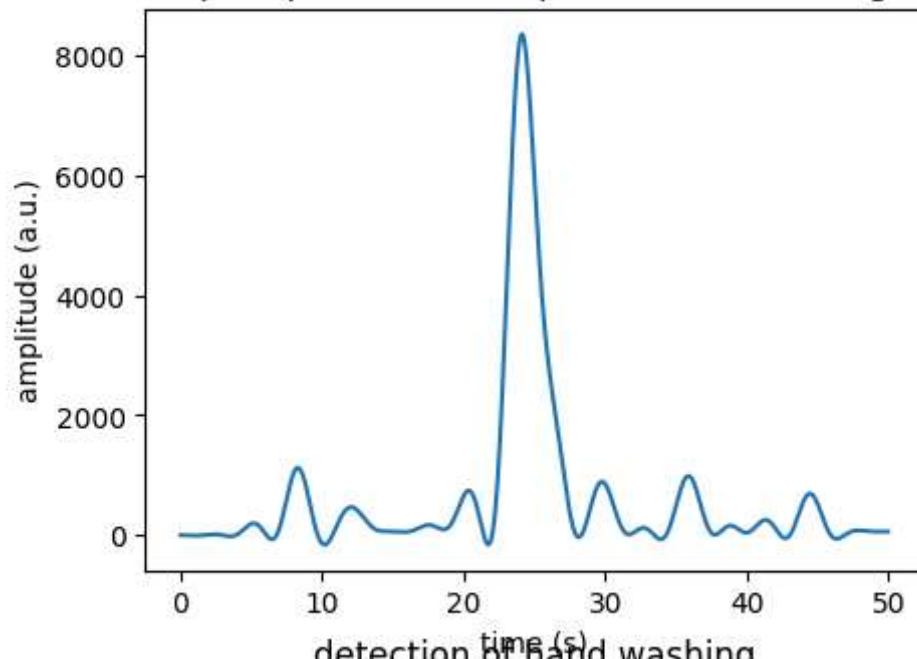
Analogic limit of the passband frequency

```
In [9]: f_pass = 0.4
# Analogic limit of the stopband frequency
f_stop = 0.8
# Conversion into Nyquist frequency
f_pass_N = f_pass/fs*2
f_stop_N = f_stop/fs*2
# Max attenuation in passband (dB)
g_pass = 3
# Min attenuation in stopband (dB)
g_stop = 40
# Determine the order and the cutoff frequency of a butterworth filter
ord, wn = sp.buttord(f_pass_N, f_stop_N, g_pass, g_stop)
# Compute the coefficients of the filter
b, a = sp.butter(ord, wn)
# Filter the signal
x_pow = sp.filtfilt(b, a, x_bp**2)
# detection
det = x_pow > 2000
```

```
In [10]: py.figure(4, figsize=[5,8])
py.clf()
py.subplot(2,1,1)
py.plot(t, x_pow)
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('low-pass power of band-pass acceleration signal')
py.subplot(2,1,2)
py.plot(t, x_hp, label='acceleration')
py.plot(t, det*200, linewidth=2, label='detection')
py.xlabel('time (s)')
py.ylabel('amplitude (a.u.)')
py.title('detection of hand washing')
py.legend(loc='upper right')
```

```
Out[10]: <matplotlib.legend.Legend at 0x220b0ea4980>
```

low-pass power of band-pass acceleration signal



detection of hand washing

